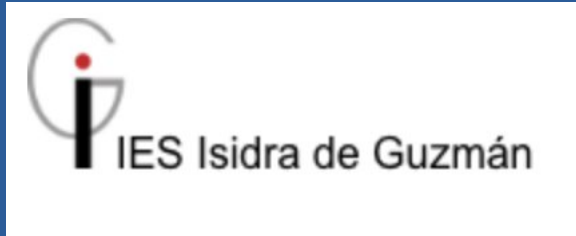


EXAMEN - Fundamentos de Programación (DAM1)

RA4 (15%) · RA5 (15%) · RA6 (15%) = porcentajes de la nota final del módulo.

3ª Evaluación



DEPARTAMENTO DE INFORMÁTICA
IES ISIDRA DE GUZMÁN (ALCALÁ DE H.)

Duración:

90 minutos

Puntuación:

RA4: RA5: RA6:

Cada RA tiene una ponderación de su puntuación de 0 a 10. Siendo nota RA ≥ 5 para poder superarlo.

Nombre:

Fecha:

30-Abril-2026 8.30am - 10am

INSTRUCCIONES

- Lee cada ejercicio completo antes de empezar.
- Nombra los archivos exactamente como se indica (mayúsculas, extensiones).
- Comenta brevemente el código cuando lo consideres necesario.
- Está permitido consultar la documentación oficial de Python así como el material teórico de las clases.
- No está permitido el plagio de código así como el uso de herramientas de IA. Ni de cualquier dispositivo inteligente como móvil, tablet, reloj, pulseras, etc. Cualquier circunstancia de las anteriores o similares dará como resultado RA4 = RA5 = RA6 = 0
- Si el alumno o alumnos han incurrido en plagio, copia o uso de IA una vez corregido el examen se tendrán que presentar a un tribunal de corrección.

EJERCICIO 1 — RA4: Programación Orientada a Objetos [3,5 puntos]

Contexto: Vas a modelar el sistema de gestión de un gimnasio.

Apartado A — Clase Cliente completa [1,2 puntos]

Crea una clase Cliente con los siguientes atributos y métodos:

- Atributos: nombre (str), edad (int), cuota_mensual (float)

Métodos:

- mostrar_info() que devuelva una cadena con este formato:

"Carlos López | 35 años | Cuota: 45.50€"

- cuota_final() que devuelva el valor de cuota_mensual (sin descuento para clientes normales).
- es_senior() que devuelva True si la edad es ≥ 60 , False en caso contrario.
- Crea una lista con al menos 3 objetos Cliente (al menos uno debe ser senior, edad ≥ 60).
- Recorre la lista mostrando la información de cada cliente y si es senior o no.

Apartado B — Herencia: ClienteVIP [1,5 puntos]

Crea la clase ClienteVIP que herede de Cliente con los siguientes requisitos:

- Añade el atributo descuento (int, porcentaje entre 0 y 100).
- Usa super() en __init__ para reutilizar el constructor padre.
- Sobrescribe mostrar_info() añadiendo al final: [VIP – X% dto.]

"Ana García | 28 años | Cuota: 45.50€ [VIP – 20% dto.]"

- Sobrescribe cuota_final() para que calcule y devuelva la cuota con el descuento aplicado.
- Añade un método exclusivo zona_vip() que devuelva el mensaje: "Acceso a zona VIP disponible"

Demuestra el uso de herencia:

- Crea una lista mixta con 4 objetos, 2 de Cliente y 2 de ClienteVIP.
- Recórrela mostrando para cada uno: mostrar_info() Y cuota_final()
- Crea un ClienteVIP específico y llama a su método zona_vip() para demostrar que funciona.

Apartado C — Interfaz abstracta [0,8 punto]

Usando el módulo abc:

- Crea la clase abstracta Servicio con el método abstracto describir().
- Implementala en dos clases concretas:
 - Entrenamiento: descripción sobre sesiones personalizadas
 - Nutricion: descripción sobre planes alimenticios
- Crea una lista con ambos servicios y recórrela llamando a describir() en cada uno.

EJERCICIO 2 — RA5: Ficheros CSV y JSON [3,5 puntos]

Contexto: Gestión de las clases y configuración del gimnasio usando ficheros.

Apartado A — Fichero CSV [1,5 puntos]

1. Crea el fichero `clases_gym.csv` usando los siguientes datos:

```
datos = [  
    ["actividad", "instructor", "plazas", "precio"],  
    ["Yoga", "Laura", 15, 12.00],  
    ["Spinning", "Pedro", 20, 10.00],  
    ["Pilates", "Laura", 12, 14.00],  
    ["Zumba", "María", 25, 8.00],  
    ["CrossFit", "Pedro", 10, 18.00],  
]
```

2. Lee el fichero con `csv.DictReader` y muestra por pantalla solo las actividades que cumplan las dos condiciones:

- plazas > 12
- precio < 15.00 €

Apartado B — Fichero JSON [1 punto]

1. Crea el fichero `config_gym.json` con estos datos iniciales:

```
config = {  
    "nombre_gym": "FitZone",  
    "horario_apertura": "07:00",  
    "horario_cierre": "22:00",  
    "numero_vestuarios": 4  
}
```

2. Lee el fichero, cambia `horario_cierre` a "23:00" y guarda el fichero actualizado.
3. Muestra la configuración final por pantalla.

Apartado C — Exportar a TXT [1 punto]

A partir de `clases_gym.csv` (léelo de nuevo), genera el fichero `resumen_clases.txt` con este formato exacto:

Yoga - Laura - 15 plazas

Spinning - Pedro - 20 plazas

Pilates - Laura - 12 plazas

Zumba - María - 25 plazas

CrossFit - Pedro - 10 plazas

Total de plazas disponibles: 82

EJERCICIO 3 — RA6: Base de Datos SQLite [3,5 puntos]

Contexto: Gestiona los socios del gimnasio en una base de datos SQLite.

Crea el archivo `bd_gym.py`. Usa SIEMPRE consultas parametrizadas con ?

Apartado A — Crear la base de datos y la tabla [0,5 puntos]

- Conéctate a `gimnasio.db` (se crea sola si no existe).
- Crea la tabla `socios` (si no existe) con esta estructura:

```
id      INTEGER PRIMARY KEY AUTOINCREMENT
nombre  TEXT
actividad TEXT
meses_inscrito INTEGER
cuota   REAL
UNIQUE(nombre, actividad)
```

Apartado B — Insertar datos [1 punto]

Inserta los siguientes socios de una sola vez usando `cursor.executemany()`:

```
socios = [
    ("Carlos", "Yoga", 6, 45.00),
    ("Ana", "Spinning", 3, 40.00),
    ("Laura", "CrossFit", 12, 55.00),
    ("Pedro", "Yoga", 2, 45.00),
    ("Marta", "Pilates", 8, 50.00),
]
```

- Después, intenta insertar a Carlos en Yoga otra vez dentro de un bloque `try/except` capturando `sqlite3.IntegrityError`. Muestra un mensaje indicando que el socio ya existe.

Apartado C — Consulta parametrizada [0,5 puntos]

Define estas dos variables y realiza una consulta `SELECT` parametrizada que devuelva los socios que cumplan ambas condiciones:

```
actividad_buscar = "Yoga"
meses_minimo = 3
```

- Muestra los resultados con `fetchall()` usando un bucle.

Apartado D — Actualización y borrado [1 punto]

- Actualiza la cuota de Laura a 60.00 €.
- Borra al socio Pedro por nombre.
- Borra todos los socios con `meses_inscrito < 4`.

Recuerda hacer `conn.commit()` después de cada operación de escritura.

Apartado E — Mostrar registros y cerrar [0,5 puntos]

- Muestra todos los registros restantes iterando sobre `cursor.fetchall()`.
- Cierra la conexión con `conn.close()`.

EJERCICIO 1 — RA4 (cada apartado normalizado a 10 al corregir)

Ej.	Criterio	Puntos	Nota
1A	Clase Cliente: atributos correctos (nombre, edad, cuota_mensual)	0.25 pts	
1A	Método mostrar_info() con formato exacto	0.25 pts	
1A	Método cuota_final() devuelve cuota_mensual	0.2 pts	
1A	Método es_senior() funciona correctamente (≥ 60)	0.25 pts	
1A	Lista con ≥ 3 clientes (al menos 1 senior) + bucle mostrando info y senior	0.25 pts	
1B	ClienteVIP hereda; super(); atributo descuento	0.4 pts	
1B	mostrar_info() sobrescrito con [VIP – X% dto.]	0.3 pts	
1B	cuota_final() sobrescrito calculando descuento correctamente	0.3 pts	
1B	Método exclusivo zona_vip() devuelve mensaje correcto	0.2 pts	
1B	Lista mixta + bucle mostrando mostrar_info() Y cuota_final() de cada uno	0.3 pts	
1C	Clase abstracta Servicio con @abstractmethod describir()	0.3 pts	
1C	Entrenamiento y Nutricion implementan describir() correctamente	0.2 pts	
1C	Lista con ambos servicios + bucle llamando a describir()	0.3 pts	
—	TOTAL EJERCICIO 1	3,5 pts	

EJERCICIO 2 — RA5 (cada apartado normalizado a 10 al corregir)

Ej.	Criterio	Puntos	Nota
2A	Crea clases_gym.csv con csv.writer y los datos proporcionados	0,5 pts	

2A	Lee con DictReader y filtra correctamente (plazas > 12 Y precio < 15)	0,7 pts	
2A	Conversión de tipos correcta (float/int) antes de comparar	0,4 pts	
2B	Crea config_gym.json con json.dump e indent	0,3 pts	
2B	Lee, modifica horario_cierre y guarda el JSON actualizado	0,5 pts	
2B	Muestra el diccionario final por pantalla	0,1 pts	
2C	Lee el CSV correctamente y genera cada línea con el formato pedido	0,6 pts	
2C	Calcula e imprime el total de plazas correctamente al final del fichero	0,4 pts	
—	TOTAL EJERCICIO 2	3,5 pts	

EJERCICIO 3 — RA6 (cada apartado normalizado a 10 al corregir)

Ej.	Criterio	Puntos	Nota
3A	Conexión a gimnasio.db + tabla socios con todos los campos y UNIQUE	0,4 pts	
3B	Inserta los 5 socios con executemany y parámetros correctamente	0,6 pts	
3B	try/except con IntegrityError + mensaje de error descriptivo	0,5 pts	
3C	SELECT con dos parámetros (? y ?) y fetchall(); muestra filas correctas	0,6 pts	
3D	UPDATE correcto de la cuota de Laura con parámetros	0,35 pts	
3D	DELETE de Pedro por nombre con parámetros	0,35 pts	
3D	DELETE con condición meses < 4 + commit() en cada operación de escritura	0,4 pts	
3E	SELECT * + fetchall() + bucle mostrando todos los registros restantes	0,2 pts	

3E	conn.close() al final del script	0,1 pts	
—	TOTAL EJERCICIO 3	3,5 pts	

NOTA TOTAL	RA4 / RA5 / RA6
-------------------	------------------------

CRITERIOS GENERALES DE CORRECCIÓN

- Cada apartado es independiente; un error en uno no penaliza en los demás. Pero tiene que funcionar
- Aunque no estén todos los apartados, el ejercicio tiene que compilar
- Criterios de puntuación de cada apartado que se multiplica por:
 - 0 pts = No ejecuta, da errores críticos, o falta el apartado completo
 - 0.5 pts = Ejecuta pero:
 - Falta algún elemento pedido (ej: falta super(), falta un método)
 - Formato incorrecto (salida no coincide)
 - Lógica parcialmente correcta
 - 1 pt = Todo correcto según la rúbrica